

Registro de Desenvolvimento do Ambiente de Simulação ROS 2 + Gazebo

Disciplina de Modelagem e Controle de Robôs

Versão 0.1 – 10 de setembro de 2026

Natureza deste documento. Este é um registro vivo do projeto. Ele deve ser atualizado ao longo do desenvolvimento para documentar decisões pedagógicas, arquitetura, configuração do ambiente, testes executados, resultados observados e próximos passos. O arquivo-fonte em L^AT_EX deve ser mantido junto ao repositório do projeto e recompilado a cada marco relevante.

Sumário

1	Objetivo do projeto	3
2	Estratégia pedagógica em dois níveis	3
2.1	Manipulador 2R: aprendizagem da ferramenta	3
2.2	Manipuladores Universal Robots: trabalho avaliativo	3
3	Arquitetura funcional desejada	4
4	Ambiente computacional verificado	4
5	Validação inicial da infraestrutura	5
5.1	Teste oficial do <code>gz_ros2_control</code>	5
5.2	UR5e no Gazebo	5
5.3	Configuração local para comando direto de torque	5
5.4	Teste de torque no UR5e	5
6	Convenção de ordenação das variáveis	6
7	Aquisição, visualização e geração de resultados	6
8	Container e distribuição aos alunos	6
9	Decisões consolidadas até esta versão	7
10	Próximos passos	7
11	Comandos de referência já utilizados	7

11.1 Ativação do ambiente ROS	7
11.2 Abertura do UR5e com controlador de esforço	8
11.3 Inspeção de controladores e interfaces	8
11.4 Teste de comando por torque	8
12 Histórico de versões	8

1 Objetivo do projeto

Construir uma bancada virtual de experimentos para a disciplina de Modelagem e Controle de Robôs. O foco da disciplina permanece na modelagem e no controle; ROS 2 e Gazebo devem funcionar como infraestrutura experimental, e não como o principal objeto de aprendizagem.

O fluxo pedagógico pretendido é

$$\boxed{\text{Teoria}} \longrightarrow \boxed{\text{MATLAB}} \longrightarrow \boxed{\text{ROS 2/Gazebo}} \longrightarrow \boxed{\text{validação e análise}}. \quad (1)$$

Os alunos deverão ser capazes de:

- desenvolver modelos cinemáticos e validá-los contra o simulador;
- desenvolver modelos dinâmicos e comparar suas previsões com a física do Gazebo;
- implementar controladores no MATLAB e posteriormente executá-los sobre a planta simulada;
- registrar, visualizar e analisar quantitativamente os resultados experimentais.

2 Estratégia pedagógica em dois níveis

2.1 Manipulador 2R: aprendizagem da ferramenta

O manipulador 2R será o ambiente introdutório. Seu objetivo é permitir que os alunos aprendam a cadeia ROS 2–Gazebo com um sistema cuja cinemática e dinâmica são suficientemente simples para que erros da ferramenta possam ser separados de erros de modelagem.

A sequência de aprendizagem prevista inclui:

- execução do simulador;
- leitura de `/joint_states`;
- identificação de juntas, links, tópicos e frames;
- comando direto por esforço/torque;
- validação de cinemática e Jacobiano;
- registro com `rosvbag2`;
- visualização com PlotJuggler;
- geração de gráficos padronizados.

2.2 Manipuladores Universal Robots: trabalho avaliativo

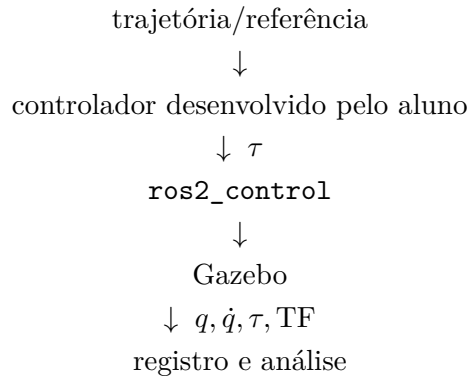
O trabalho avaliativo deverá utilizar um manipulador industrial de 6 GDL da família Universal Robots. A arquitetura inicial foi validada com o UR5e.

Uma alternativa pedagogicamente interessante, atualmente em avaliação, é dividir os grupos entre **UR3e** e **UR5e**. Ambos pertencem à mesma família de manipuladores de 6 GDL e podem ser tratados com a mesma infraestrutura de software, mas apresentam dimensões e parâmetros físicos diferentes. Isso permite propor trabalhos estruturalmente equivalentes sem fazer com que todos os grupos usem exatamente a mesma planta.

Para reduzir a carga algébrica da dinâmica quando necessário, poderá ser utilizada uma variante com as três juntas do punho fixadas, mantendo ativos os três primeiros graus de liberdade. Os elos do punho não devem ser eliminados fisicamente: suas massas e inércias continuam contribuindo para a dinâmica do braço.

3 Arquitetura funcional desejada

A arquitetura conceitual é



A atuação deve ser preferencialmente por esforço, de modo que a lei de controle permaneça explícita no código do aluno. Em particular, deseja-se evitar um controlador de posição interno que esconda a dinâmica relevante para a disciplina.

Para um controlador por torque calculado, por exemplo,

$$\tau = M(q)v + C(q, \dot{q})\dot{q} + g(q), \quad (2)$$

a expressão acima deve ser calculada pelo código do aluno e o ROS deve apenas encaminhar o vetor de torques ao simulador.

4 Ambiente computacional verificado

O PC do laboratório foi inspecionado e apresentou a seguinte configuração relevante:

Item	Situação verificada
Sistema operacional	Ubuntu 24.04 LTS
ROS 2	Jazzy, instalado em /opt/ros/jazzy
Gazebo	Gazebo Sim 8.10.0, família Harmonic
Integração ROS–Gazebo	ros_gz e gz_ros2_control disponíveis
Visualização	RViz2 disponível
Build e desenvolvimento	colcon, Git e Python 3.12 disponíveis
GPU	NVIDIA RTX 5090 com 32 GB
PlotJuggler	Instalado durante a preparação do ambiente
Simulação Universal Robots	ur_simulation_gz instalado

Por decisão de segurança operacional em uma máquina compartilhada, o ambiente ROS **não foi adicionado ao .bashrc**. Em cada novo terminal usado para o projeto é executado manualmente:

```
source /opt/ros/jazzy/setup.bash
```

Essa escolha evita alterações persistentes no ambiente dos demais usuários.

5 Validação inicial da infraestrutura

5.1 Teste oficial do `gz_ros2_control`

Foi executado o exemplo oficial de controle por esforço. O Gazebo abriu corretamente, o controlador foi carregado e os estados das juntas foram observados via ROS 2. Também foi verificado que `/joint_states` publica continuamente posição, velocidade e esforço.

Esse teste confirmou a cadeia básica

$$\boxed{\text{Gazebo}} \leftrightarrow \boxed{\text{gz_ros2_control}} \leftrightarrow \boxed{\text{ROS 2}}. \quad (3)$$

5.2 UR5e no Gazebo

O pacote `ur_simulation_gz` foi executado com o UR5e. O launch utilizado aceita, entre outros, os argumentos `ur_type`, `controllers_file`, `initial_joint_controller`, `gazebo_gui` e `world_file`.

O modelo expôs, para as seis juntas, interfaces de comando de posição, velocidade e esforço. As interfaces de estado incluem também posição, velocidade e esforço.

5.3 Configuração local para comando direto de torque

Foi criada uma configuração local, sem modificar os arquivos oficiais em `/opt/ros`, no diretório

```
~/robot_control_lab/config/ur_controllers_effort.yaml
```

A configuração utiliza

```
effort_controllers/JointGroupEffortController
```

para as juntas, na ordem física adotada pelo projeto:

1. `shoulder_pan_joint`
2. `shoulder_lift_joint`
3. `elbow_joint`
4. `wrist_1_joint`
5. `wrist_2_joint`
6. `wrist_3_joint`

Ao iniciar o UR5e com essa configuração, o `forward_effort_controller` ficou ativo e as seis interfaces `effort` ficaram marcadas como `claimed`.

5.4 Teste de torque no UR5e

Foi publicado continuamente o vetor

$$\tau = [5, 0, 0, 0, 0, 0]^T \text{ N m}. \quad (4)$$

O tópico de comando confirmou o vetor publicado e `/joint_states` registrou exatamente 5.0 no esforço correspondente à `shoulder_pan_joint`. Assim, ficou validado que um torque calculado externamente pode ser aplicado diretamente ao modelo do UR5e no Gazebo.

6 Convenção de ordenação das variáveis

Foi observado que a ordem de juntas publicada em `/joint_states` não necessariamente coincide com a ordem física usada nas equações do robô. No teste do UR5e, a mensagem foi recebida na ordem

```
elbow_joint
shoulder_lift_joint
shoulder_pan_joint
wrist_1_joint
wrist_2_joint
wrist_3_joint
```

A infraestrutura da disciplina **não deve depender dessa ordem**. O nó de interface deve associar cada valor ao nome da junta e, imediatamente após a leitura, reorganizar os estados na convenção matemática

$$q = [q_1, q_2, \dots, q_n]^T, \quad \dot{q} = [\dot{q}_1, \dot{q}_2, \dots, \dot{q}_n]^T. \quad (5)$$

Para o UR5e, a convenção interna será

```
q1 ↔ shoulder_pan_joint,
q2 ↔ shoulder_lift_joint,
q3 ↔ elbow_joint,
q4 ↔ wrist_1_joint,
q5 ↔ wrist_2_joint,
q6 ↔ wrist_3_joint.
```

A mesma filosofia será usada no 2R, com $q = [q_1, q_2]^T$ e $\tau = [\tau_1, \tau_2]^T$.

7 Aquisição, visualização e geração de resultados

A estratégia planejada combina três ferramentas:

- **PlotJuggler**: visualização interativa em tempo real durante os experimentos;
- **rosbag2**: registro reprodutível dos sinais de cada ensaio;
- **Python/matplotlib**: geração automática de gráficos e métricas padronizadas para os relatórios.

Os resultados desejados incluem, conforme a atividade:

- $q_d(t)$ e $q(t)$;
- erro $e(t)$;
- $\dot{q}(t)$;
- torque $\tau(t)$;
- posição e/ou orientação cartesiana do efetuador;
- métricas como RMSE, máximo erro absoluto, esforço máximo e integrais de erro.

8 Container e distribuição aos alunos

O desenvolvimento inicial será realizado nativamente no PC do laboratório, onde a configuração ROS/Gazebo já foi validada. O container será construído **depois que a cadeia nativa estiver estável**, evitando depurar simultaneamente ROS, Gazebo, GPU, Docker e o código didático.

O container deverá oferecer:

- execução headless obrigatória;
- GUI opcional para Gazebo/PlotJuggler;
- dependências congeladas;
- comandos de alto nível para iniciar experimentos;
- volume para os arquivos modificados pelos alunos.

9 Decisões consolidadas até esta versão

1. ROS 2 e Gazebo serão infraestrutura da disciplina, não o principal conteúdo avaliado.
2. O 2R será utilizado para aprender a ferramenta e o fluxo experimental.
3. Um manipulador Universal Robots de 6 GDL será utilizado no trabalho avaliativo.
4. O comando de controle será preferencialmente por torque/esforço explícito.
5. A ordem interna das variáveis será padronizada e construída a partir dos nomes das juntas.
6. A infraestrutura não deve calcular automaticamente os modelos ou leis de controle que fazem parte do trabalho dos alunos.
7. O desenvolvimento será estabilizado nativamente antes da criação do container.
8. Alterações específicas do projeto permanecerão no diretório/repositório do projeto, evitando modificar arquivos oficiais do ROS sempre que possível.

10 Próximos passos

O próximo marco é retornar à ordem pedagógica planejada e construir o ambiente 2R. A sequência sugerida é:

1. verificar os pacotes 2R/RRBot disponíveis na instalação Jazzy;
2. executar o 2R no Gazebo;
3. garantir comando direto de torque em suas duas juntas;
4. criar um primeiro nó Python de interface que leia `/joint_states`, reorganize os estados por nome e publique τ ;
5. testar aquisição com rosbag2 e visualização com PlotJuggler;
6. implementar um primeiro controlador PD e registrar um experimento completo;
7. automatizar gráficos e métricas;
8. reutilizar a mesma arquitetura para UR3e/UR5e;
9. definir a forma final de redução para três GDL, caso seja adotada no trabalho;
10. criar e testar o container dos alunos.

11 Comandos de referência já utilizados

11.1 Ativação do ambiente ROS

```
source /opt/ros/jazzy/setup.bash
```

11.2 Abertura do UR5e com controlador de esforço

```
ros2 launch ur_simulation_gz ur_sim_control.launch.py \  
  ur_type:=ur5e \  
  controllers_file:=$HOME/robot_control_lab/config/ur_controllers_effort.yaml \  
  initial_joint_controller:=forward_effort_controller
```

11.3 Inspeção de controladores e interfaces

```
ros2 control list_controllers  
ros2 control list_hardware_interfaces  
ros2 topic echo /joint_states --once
```

11.4 Teste de comando por torque

```
ros2 topic pub -r 20 \  
  /forward_effort_controller/commands \  
  std_msgs/msg/Float64MultiArray \  
  "{data: [5.0, 0.0, 0.0, 0.0, 0.0, 0.0]}"
```

12 Histórico de versões

Versão	Data	Conteúdo
0.1	10/09/2026	Consolidação da estratégia, inventário do ambiente, validação de <code>gz_ros2_control</code> , teste do UR5e por esforço, convenção de juntas e definição dos próximos passos.